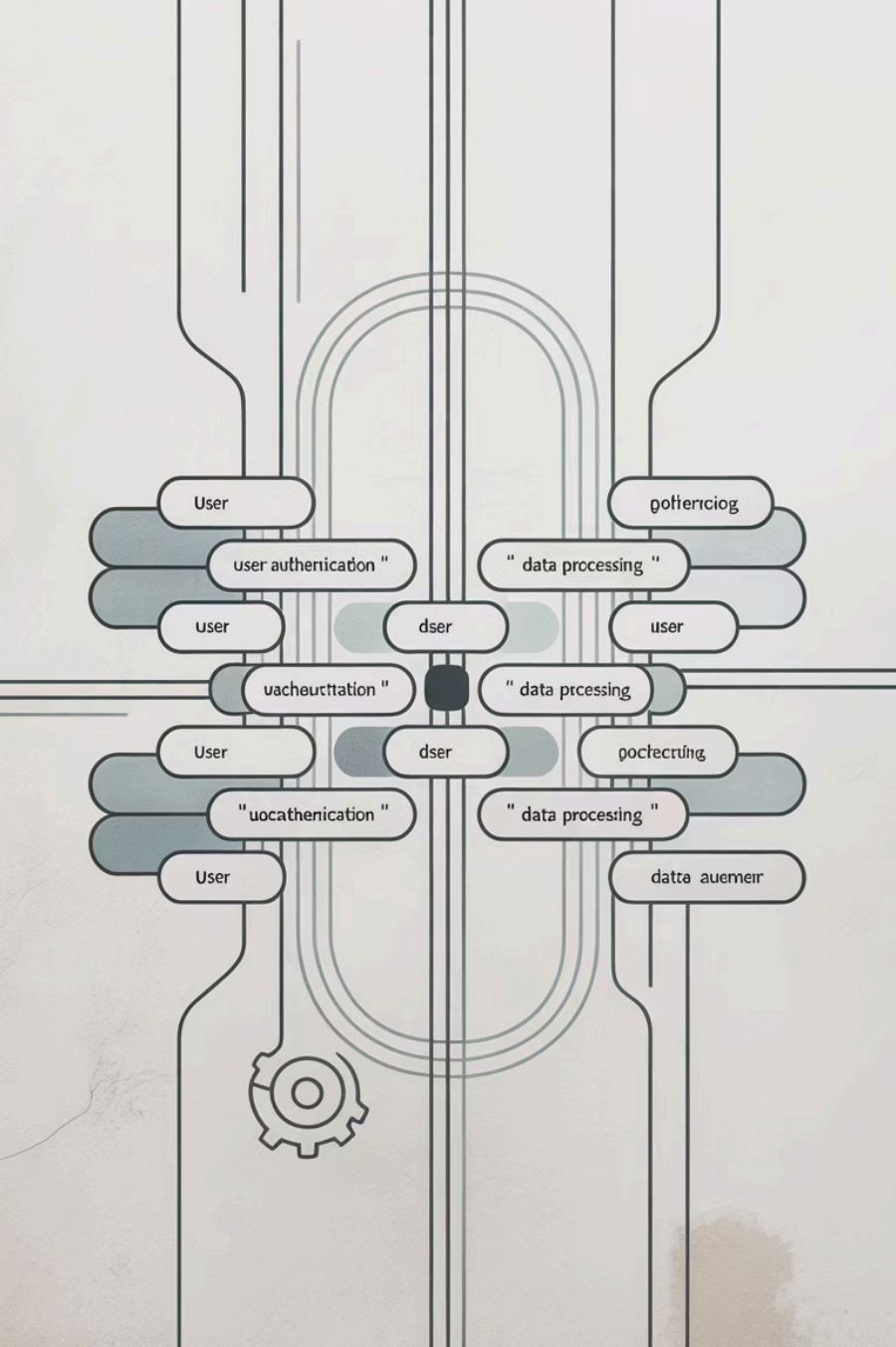




Top 5 Design Patterns in Microservices

Building Resilient and Scalable Systems

by Naresh Chaurasia

Agenda

01

API Gateway Pattern

02

Database per Service Pattern

03

Strangler Fig Pattern

04

Circuit Breaker Pattern

05

Event-Driven Pattern

API Gateway Pattern

- One entry point for all clients
- Handles routing, authentication, monitoring
- Simplifies communication between client and services

Icons/Visuals: 🗝️ 🌐 🗺️

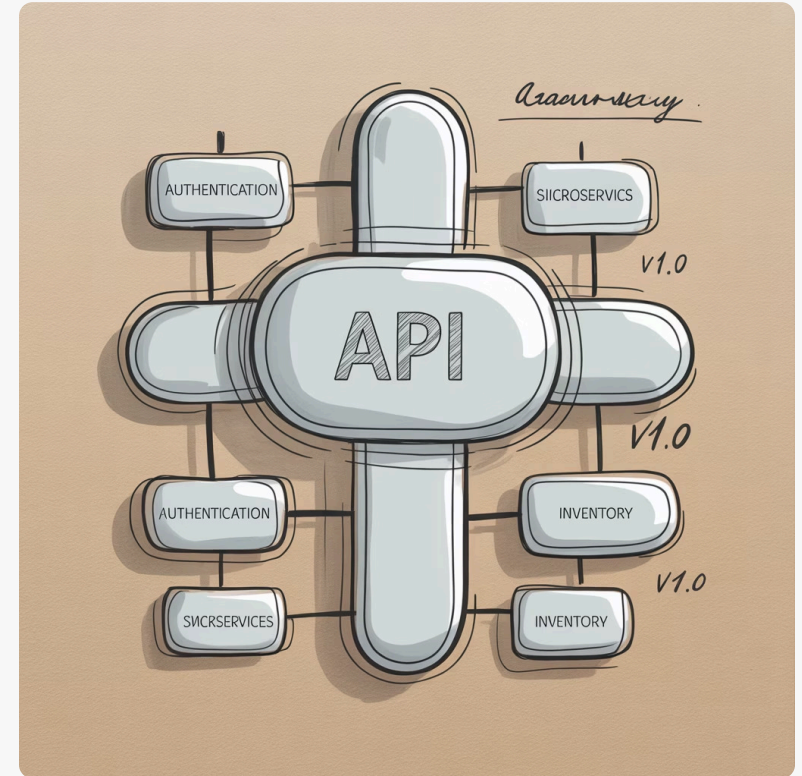


Diagram: Clients → API Gateway → Microservices

Database per Service Pattern

- Each service owns its own database
- Prevents tight coupling
- Supports independent scaling and polyglot persistence

Icons/Visuals:   

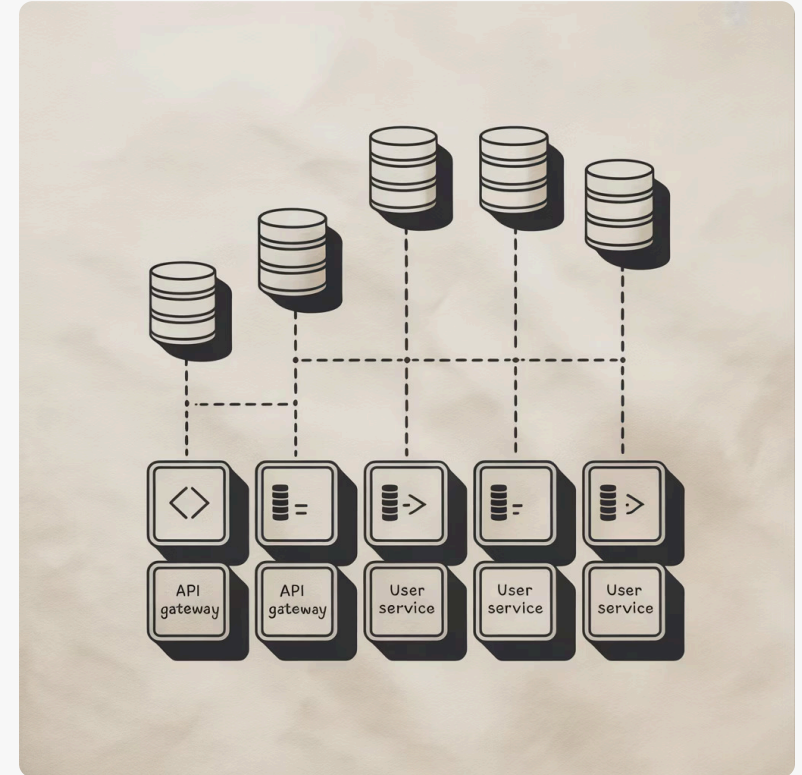


Illustration: Microservices each with own DB

Strangler Fig Pattern

- Gradually migrate from monolith to microservices
- New features go into microservices, old features remain in monolith
- Over time, monolith gets "strangled"

Icons/Visuals:   

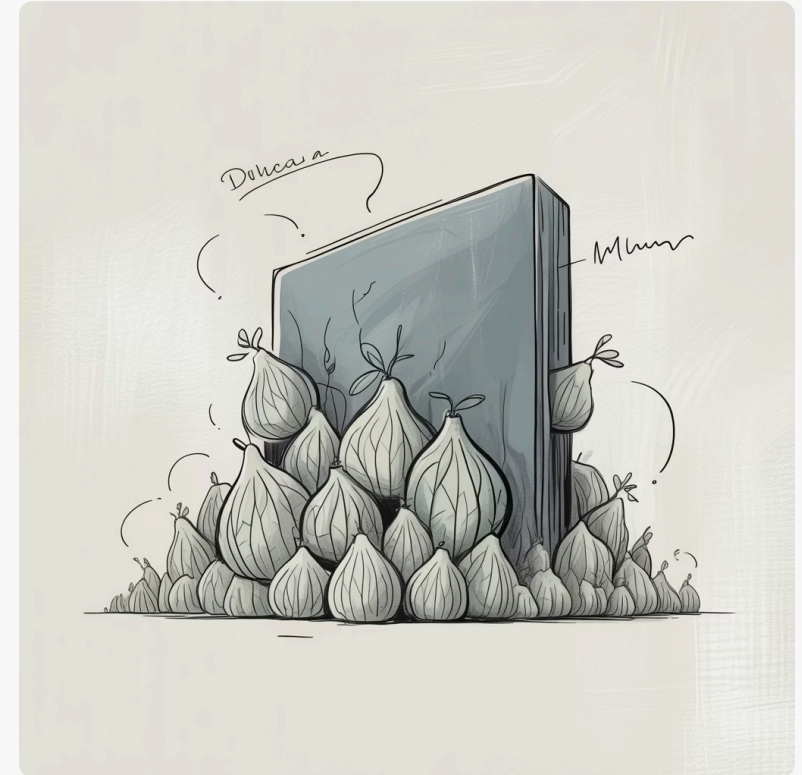


Illustration: Monolith → Microservices branching out

Circuit Breaker Pattern

- Prevents cascading failures
- Stops sending requests to failing services temporarily
- Improves fault tolerance

Icons/Visuals: ⚡️ 🔌 🚦

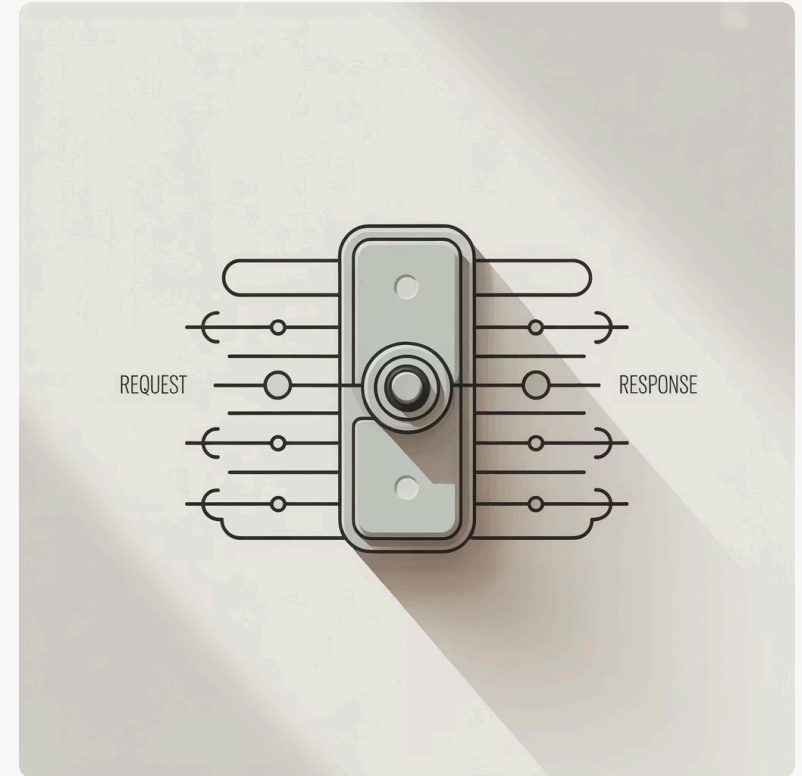


Illustration: Service request through breaker switch

Event-Driven Pattern

- Services communicate asynchronously via events
- Improves scalability and decoupling
- Useful for real-time systems

Icons/Visuals: 📢 ↺ 📧

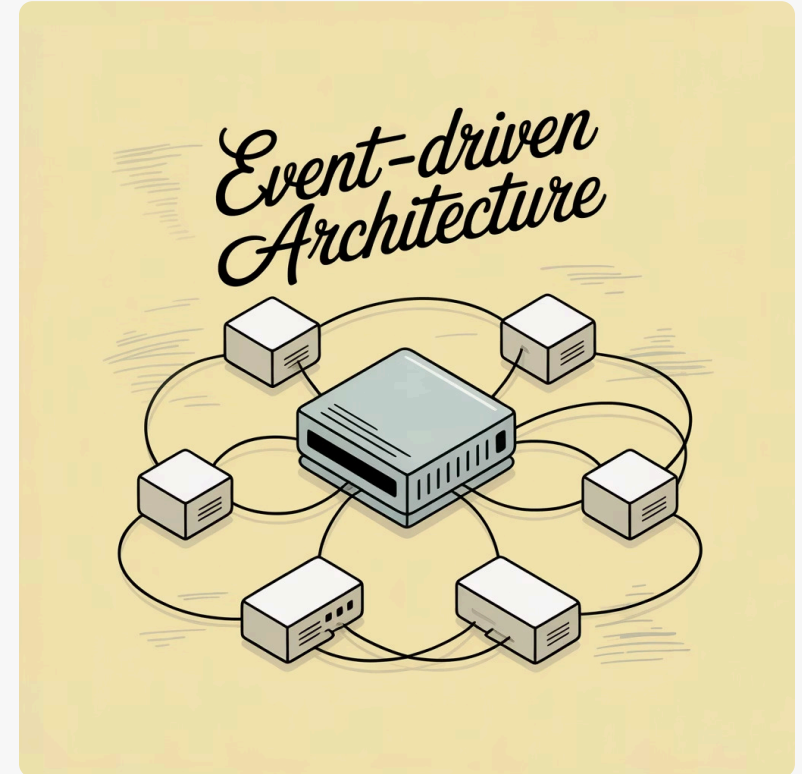


Illustration: Event broker (Kafka/RabbitMQ) with services around it

Summary

API Gateway

Single entry point

Database per Service

Independent persistence

Strangler Fig

Migration strategy

Circuit Breaker

Fault tolerance

Event-Driven

Asynchronous communication

Icons/Visuals:    | Simple checklist layout with 5 icons

Benefits of These Patterns



Enhanced Resilience

Circuit breakers and independent databases prevent system-wide failures and improve fault tolerance across your microservices architecture.



Improved Scalability

Event-driven patterns and database separation allow individual services to scale independently based on demand.



Simplified Migration

The strangler fig pattern provides a safe, gradual path from monolithic to microservices architecture without disrupting existing functionality.

Next Steps



Study Implementation

Research specific technologies and frameworks that support these patterns in your chosen tech stack.



Start Small

Begin with one pattern, such as API Gateway, to gain experience before implementing more complex patterns.



Scale Gradually

Apply lessons learnt to implement additional patterns as your microservices architecture matures and grows.